

Definitions in **Sarason/** and **R2.lean**

Lean Formalization Project

September 29, 2026

This document provides a comprehensive list of all definitions manually formalized in the **ComplexAnalysis** project, specifically within **R2.lean** and the **Sarason/** directory. For each definition, the raw Lean code is presented first, followed by its mathematical interpretation.

1 **ComplexAnalysis/R2.lean**

- **sqNorm** ($x \in \mathbb{R}^2 \rightarrow \mathbb{R}$): The squared Euclidean norm.

```
noncomputable def sqNorm (x : ℝ × ℝ) : ℝ := x.1^2 + x.2^2
```

$$\text{sqNorm}(x) = x_1^2 + x_2^2$$

- **sqDist** ($x, y \in \mathbb{R}^2 \rightarrow \mathbb{R}$): The squared Euclidean distance.

```
noncomputable def sqDist (x y : ℝ × ℝ) : ℝ :=  
  (x.1 - y.1)^2 + (x.2 - y.2)^2
```

$$\text{sqDist}(x, y) = \text{sqNorm}(x - y)$$

- **euclideanNorm** ($x \in \mathbb{R}^2 \rightarrow \mathbb{R}$): The standard Euclidean norm.

```
noncomputable def euclideanNorm (x : ℝ × ℝ) : ℝ :=  
  Real.sqrt (sqNorm x)
```

$$\text{euclideanNorm}(x) = \sqrt{x_1^2 + x_2^2}$$

- **euclideanDist** ($x, y \in \mathbb{R}^2 \rightarrow \mathbb{R}$): The Euclidean distance.

```
noncomputable def euclideanDist (x y : ℝ × ℝ) : ℝ :=  
  Real.sqrt (sqDist x y)
```

$$\text{euclideanDist}(x, y) = \sqrt{\text{sqDist}(x, y)}$$

- **LimitRtoR2** ($f : \mathbb{R} \rightarrow \mathbb{R}^2, a \in \mathbb{R}, L \in \mathbb{R}^2$):

```
def LimitRtoR2 (f : ℝ → ℝ × ℝ) (a : ℝ) (L : ℝ × ℝ) : Prop :=  
  ∀ ε > 0, ∃ δ > 0, ∀ x : ℝ, 0 < |x - a| ∧ |x - a| < δ → euclideanDist (f x) L  
    < ε
```

$$\lim_{x \rightarrow a} f(x) = L \iff \forall \epsilon > 0, \exists \delta > 0, \forall x, 0 < |x - a| < \delta \implies \text{euclideanDist}(f(x), L) < \epsilon$$

- **LimitR2toR** ($f : \mathbb{R}^2 \rightarrow \mathbb{R}, a \in \mathbb{R}^2, L \in \mathbb{R}$):

```
def LimitR2toR (f : ℝ × ℝ → ℝ) (a : ℝ × ℝ) (L : ℝ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ x : ℝ × ℝ, 0 < euclideanDist x a ∧ euclideanDist x a < δ
    → |f x - L| < ε
```

$$\lim_{x \rightarrow a} f(x) = L \iff \forall \epsilon > 0, \exists \delta > 0, \forall x \in \mathbb{R}^2, 0 < \text{euclideanDist}(x, a) < \delta \implies |f(x) - L| < \epsilon$$

- **LimitR2toR2** ($f : \mathbb{R}^2 \rightarrow \mathbb{R}^2, a \in \mathbb{R}^2, L \in \mathbb{R}^2$):

```
def LimitR2toR2 (f : ℝ × ℝ → ℝ × ℝ) (a : ℝ × ℝ) (L : ℝ × ℝ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ x : ℝ × ℝ, 0 < euclideanDist x a ∧ euclideanDist x a < δ
    → euclideanDist (f x) L < ε
```

$$\lim_{x \rightarrow a} f(x) = L \iff \forall \epsilon > 0, \exists \delta > 0, \forall x \in \mathbb{R}^2, 0 < \text{euclideanDist}(x, a) < \delta \implies \text{euclideanDist}(f(x), L) < \epsilon$$

- **ConvergesR2** ($\text{seq} : \mathbb{N} \rightarrow \mathbb{R}^2, L \in \mathbb{R}^2$):

```
def ConvergesR2 (seq : ℕ → ℝ × ℝ) (L : ℝ × ℝ) : Prop :=
  ∀ ε > 0, ∃ N : ℕ, ∀ n ≥ N, euclideanDist (seq n) L < ε
```

$$\lim_{n \rightarrow \infty} \text{seq}(n) = L \iff \forall \epsilon > 0, \exists N \in \mathbb{N}, \forall n \geq N, \text{euclideanDist}(\text{seq}(n), L) < \epsilon$$

- **HasFDerivAt_R2_eps** ($u, v : \mathbb{R}^2 \rightarrow \mathbb{R}, u_{x0}, u_{y0}, v_{x0}, v_{y0} \in \mathbb{R}, a \in \mathbb{R}^2$): Defines the Fréchet derivative of $F(x, y) = (u(x, y), v(x, y))$ at a . Let $L(h) = \begin{pmatrix} u_{x0} & u_{y0} \\ v_{x0} & v_{y0} \end{pmatrix} h$.

```
def HasFDerivAt_R2_eps (u v : ℝ × ℝ → ℝ) (ux0 uy0 vx0 vy0 : ℝ) (a : ℝ × ℝ) :
  Prop :=
  LimitR2toR (fun h =>
    euclideanNorm (
      (u (a.1 + h.1, a.2 + h.2) - u a) - (ux0 * h.1 + uy0 * h.2),
      (v (a.1 + h.1, a.2 + h.2) - v a) - (vx0 * h.1 + vy0 * h.2)
    ) / euclideanNorm h
  ) (0, 0) 0
```

$$\lim_{h \rightarrow 0} \frac{\left\| F(a + h) - F(a) - \begin{pmatrix} u_{x0} & u_{y0} \\ v_{x0} & v_{y0} \end{pmatrix} h \right\|}{\|h\|} = 0$$

$$\lim_{(h_1, h_2) \rightarrow (0, 0)} \frac{\left\| \begin{pmatrix} u(a_1 + h_1, a_2 + h_2) - u(a_1, a_2) \\ v(a_1 + h_1, a_2 + h_2) - v(a_1, a_2) \end{pmatrix} - \begin{pmatrix} u_{x0} & u_{y0} \\ v_{x0} & v_{y0} \end{pmatrix} \begin{pmatrix} h_1 \\ h_2 \end{pmatrix} \right\|}{\|(h_1, h_2)\|} = 0$$

- **DifferentiableAt_R2_eps** ($u, v : \mathbb{R}^2 \rightarrow \mathbb{R}, a \in \mathbb{R}^2$):

```
def DifferentiableAt_R2_eps (u v : ℝ × ℝ → ℝ) (a : ℝ × ℝ) : Prop :=
  ∃ (ux0 uy0 vx0 vy0 : ℝ), HasFDerivAt_R2_eps u v ux0 uy0 vx0 vy0 a
```

$$\exists u_{x0}, u_{y0}, v_{x0}, v_{y0} \in \mathbb{R}, \text{HasFDerivAt_R2_eps}(u, v, u_{x0}, u_{y0}, v_{x0}, v_{y0}, a)$$

- **HasDerivAt_RtoR2_eps** ($f : \mathbb{R} \rightarrow \mathbb{R}^2, f'_0 \in \mathbb{R}^2, t_0 \in \mathbb{R}$): Defines the derivative of a parametric curve in $\mathbb{R}^2$.

```
def HasDerivAt_RtoR2_eps (f : ℝ → ℝ × ℝ) (f'0 : ℝ × ℝ) (t0 : ℝ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ t : ℝ, 0 < |t - t0| ∧ |t - t0| < δ →
    euclideanDist (f t) (f t0 + (f'0.1 * (t - t0), f'0.2 * (t - t0))) < ε * |t - t0|
```

$$\lim_{t \rightarrow t_0} (f(t) - f(t_0) - (t - t_0)f'_0) = 0$$

- **HasPartialDerivX_R2_eps** ($u : \mathbb{R}^2 \rightarrow \mathbb{R}$, $ux_0 \in \mathbb{R}$, $p_0 \in \mathbb{R}^2$):

```
def HasPartialDerivX_R2_eps (u : ℝ × ℝ → ℝ) (ux₀ : ℝ) (p₀ : ℝ × ℝ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ x : ℝ, 0 < |x - p₀.1| ∧ |x - p₀.1| < δ →
    |(u (x, p₀.2) - u p₀) / (x - p₀.1) - ux₀| < ε
```

$$\lim_{x \rightarrow p_{0,x}} \frac{u(x, p_{0,y}) - u(p_0)}{x - p_{0,x}} = ux_0$$

- **HasPartialDerivY_R2_eps** ($u : \mathbb{R}^2 \rightarrow \mathbb{R}$, $uy_0 \in \mathbb{R}$, $p_0 \in \mathbb{R}^2$):

```
def HasPartialDerivY_R2_eps (u : ℝ × ℝ → ℝ) (uy₀ : ℝ) (p₀ : ℝ × ℝ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ y : ℝ, 0 < |y - p₀.2| ∧ |y - p₀.2| < δ →
    |(u (p₀.1, y) - u p₀) / (y - p₀.2) - uy₀| < ε
```

```
-- A real-valued function of two real variables is harmonic on an open set G
if its first and second partial derivatives are continuous and satisfy Laplace
's equation. -/
```

$$\lim_{y \rightarrow p_{0,y}} \frac{u(p_{0,x}, y) - u(p_0)}{y - p_{0,y}} = uy_0$$

- **Harmonic_R2_eps** ($u : \mathbb{R}^2 \rightarrow \mathbb{R}$, $G \subset \mathbb{R}^2$):

```
def Harmonic_R2_eps (u : ℝ × ℝ → ℝ) (G : Set (ℝ × ℝ)) : Prop :=
  IsOpen G ∧
  ∃ (ux uy uxx uxy uyx uyy : ℝ × ℝ → ℝ),
    (∀ p ∈ G, HasPartialDerivX_R2_eps u (ux p) p) ∧
    (∀ p ∈ G, HasPartialDerivY_R2_eps u (uy p) p) ∧
    (ContinuousOn ux G) ∧
    (ContinuousOn uy G) ∧
    (∀ p ∈ G, HasPartialDerivX_R2_eps ux (uxx p) p) ∧
    (∀ p ∈ G, HasPartialDerivY_R2_eps ux (uxy p) p) ∧
    (∀ p ∈ G, HasPartialDerivX_R2_eps uy (uyx p) p) ∧
    (∀ p ∈ G, HasPartialDerivY_R2_eps uy (uyy p) p) ∧
    (ContinuousOn uxx G) ∧
    (ContinuousOn uxy G) ∧
    (ContinuousOn uyx G) ∧
    (ContinuousOn uyy G) ∧
    (∀ p ∈ G, uxx p + uyy p = 0)
```

First and second partials continuous on G , $\Delta u = u_{xx} + u_{yy} = 0$

2 ComplexAnalysis/Sarason/Definitions.lean

- **HasLimitAt_eps** ($f : \mathbb{C} \rightarrow \mathbb{C}$, $L \in \mathbb{C}$, $z_0 \in \mathbb{C}$):

```
def HasLimitAt_eps (f : ℂ → ℂ) (L : ℂ) (z₀ : ℂ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ z, 0 < ||z - z₀|| ∧ ||z - z₀|| < δ → ||f z - L|| < ε

-- Weierstrass definition of a limit at infinity.
```

$$\lim_{z \rightarrow z_0} f(z) = L$$

- **HasLimitAtInf_eps** ($f : \mathbb{C} \rightarrow \mathbb{R}$, $L \in \mathbb{R}$):

```

def HasLimitAtInf_eps (f : ℂ → ℝ) (L : ℝ) : Prop :=
  ∀ ε > 0, ∃ M > 0, ∀ z, ||z|| > M → |f z - L| < ε

-- Weierstrass definition of having a complex derivative at z₀.

```

$$\lim_{|z| \rightarrow \infty} f(z) = L$$

- **HasDerivAt_eps** ($f : \mathbb{C} \rightarrow \mathbb{C}$, $f'_0 \in \mathbb{C}$, $z_0 \in \mathbb{C}$): Complex differentiability at a point.

```

def HasDerivAt_eps (f : ℂ → ℂ) (f'₀ : ℂ) (z₀ : ℂ) : Prop :=
  HasLimitAt_eps (fun z => (f z - f z₀) / (z - z₀)) f'₀ z₀

-- Weierstrass definition of complex differentiability at z₀.

```

$$\lim_{z \rightarrow z_0} \frac{f(z) - f(z_0)}{z - z_0} = f'_0$$

- **DifferentiableAt_eps** ($f : \mathbb{C} \rightarrow \mathbb{C}$, $z_0 \in \mathbb{C}$):

```

def DifferentiableAt_eps (f : ℂ → ℂ) (z₀ : ℂ) : Prop :=
  ∃ f'₀, HasDerivAt_eps f f'₀ z₀

-- Weierstrass definition of partial derivative with respect to x.

```

f is differentiable at z_0

- **HasPartialDerivX_C_to_R_eps** / **HasPartialDerivY_C_to_R_eps**: Partial derivatives of a real-valued function of a complex variable (identifying $\mathbb{C} \cong \mathbb{R}^2$).

```

def HasPartialDerivX_C_to_R_eps (u : ℂ → ℝ) (ux₀ : ℝ) (z₀ : ℂ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ x : ℝ, 0 < |x - z₀.re| ∧ |x - z₀.re| < δ →
    |(u ((x : ℂ) + (z₀.im : ℂ) * I) - u z₀) / (x - z₀.re) - ux₀| < ε

-- Weierstrass definition of partial derivative with respect to y.

```

$$\frac{\partial u}{\partial x}(z_0) = u_{x0}$$

- **HasPartialDerivY_C_to_R_eps**

```

def HasPartialDerivY_C_to_R_eps (u : ℂ → ℝ) (uy₀ : ℝ) (z₀ : ℂ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ y : ℝ, 0 < |y - z₀.im| ∧ |y - z₀.im| < δ →
    |(u ((z₀.re : ℂ) + (y : ℂ) * I) - u z₀) / (y - z₀.im) - uy₀| < ε

-- Equivalence theorem between Mathlib's limit at a point (using punctured
  neighborhoods) and Weierstrass epsilon-delta limit.

```

$$\frac{\partial u}{\partial y}(z_0) = u_{y0}$$

- **deriv_eps** ($f : \mathbb{C} \rightarrow \mathbb{C}$, $z_0 \in \mathbb{C}$, $h : \text{DifferentiableAt_eps } f \ z_0$): The classical derivative value $f'(z_0)$ extracted via choice.

```

noncomputable def deriv_eps (f : ℂ → ℂ) (z₀ : ℂ) (h : DifferentiableAt_eps f
  z₀) : ℂ :=
  Classical.choose h

```

$$\text{deriv_eps}(f, z_0) = f'(z_0)$$

- **HasDerivAt_R_to_C_eps** ($f : \mathbb{R} \rightarrow \mathbb{C}$, $f'_0 \in \mathbb{C}$, $t_0 \in \mathbb{R}$): Derivative of a complex-valued path parameterized by a real variable.

```
def HasDerivAt_R_to_C_eps (f : ℝ → ℂ) (f'_0 : ℂ) (t_0 : ℝ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ t : ℝ, 0 < |t - t_0| ∧ |t - t_0| < δ → ||(f t - f t_0) / ((t
    - t_0 : ℝ) : ℂ) - f'_0|| < ε
```

$$\lim_{t \rightarrow t_0} \frac{f(t) - f(t_0)}{t - t_0} = f'_0$$

- **DifferentiableAt_R_to_C_eps** ($f : \mathbb{R} \rightarrow \mathbb{C}$, $t_0 \in \mathbb{R}$):

```
def DifferentiableAt_R_to_C_eps (f : ℝ → ℂ) (t_0 : ℝ) : Prop :=
  ∃ f'_0, HasDerivAt_R_to_C_eps f f'_0 t_0

/-- The classical derivative value extracted via choice, or 0 if not
    differentiable. -/
```

$$\exists f'_0 \in \mathbb{C}, \text{HasDerivAt_R_to_C_eps}(f, f'_0, t_0)$$

- **deriv_R_to_C_eps** ($f : \mathbb{R} \rightarrow \mathbb{C}$, $t_0 \in \mathbb{R}$):

```
noncomputable def deriv_R_to_C_eps (f : ℝ → ℂ) (t_0 : ℝ) : ℂ :=
  if h : DifferentiableAt_R_to_C_eps f t_0 then Classical.choose h else 0
```

$$\text{deriv_R_to_C_eps}(f, t_0) = \begin{cases} f'(t_0) & \text{if differentiable} \\ 0 & \text{otherwise} \end{cases}$$

- **HolomorphicAt_eps** ($f : \mathbb{C} \rightarrow \mathbb{C}$, $z_0 \in \mathbb{C}$): f is complex-differentiable in some open neighborhood around z_0 .

```
def HolomorphicAt_eps (f : ℂ → ℂ) (z_0 : ℂ) : Prop :=
  DifferentiableAt_eps f z_0

/-- A function is holomorphic on a set if it is complex differentiable at
    every point of the set. -/
```

$$f \text{ is holomorphic at } z_0 \iff f \text{ is complex-differentiable on some open ball around } z_0$$

- **HolomorphicOn_eps** ($f : \mathbb{C} \rightarrow \mathbb{C}$, $G \subset \mathbb{C}$): f is holomorphic at every point in the open set G .

```
def HolomorphicOn_eps (f : ℂ → ℂ) (G : Set ℂ) : Prop :=
  ∀ z ∈ G, DifferentiableAt_eps f z
```

$$f \text{ is holomorphic on } G$$

- **TwiceContinuouslyDifferentiable_C_eps** ($f : \mathbb{C} \rightarrow \mathbb{C}$, $G \subset \mathbb{C}$): f is twice continuously differentiable on the open set G if its real and imaginary parts are.

```
def TwiceContinuouslyDifferentiable_C_eps (f : ℂ → ℂ) (G : Set ℂ) : Prop :=
  ComplexAnalysis.R2.TwiceContinuouslyDifferentiable_R2_eps (fun p => (f (p.1
    + p.2 * I)).re) {p : ℝ × ℝ | p.1 + p.2 * I ∈ G} ∧
  ComplexAnalysis.R2.TwiceContinuouslyDifferentiable_R2_eps (fun p => (f (p.1
    + p.2 * I)).im) {p : ℝ × ℝ | p.1 + p.2 * I ∈ G}
```

$$\text{Re}(f), \text{Im}(f) \in C^2(G, \mathbb{R})$$

- **Harmonic_C_eps** ($f : \mathbb{C} \rightarrow \mathbb{C}$, $G \subset \mathbb{C}$): f is harmonic on the open set G if its real and imaginary parts satisfy Laplace's equation $\Delta = 0$ on G .

```
def Harmonic_C_eps (f : ℂ → ℂ) (G : Set ℂ) : Prop :=
  ComplexAnalysis.R2.Harmonic_R2_eps (fun p => (f (p.1 + p.2 * I)).re) {p : ℝ
    × ℝ | p.1 + p.2 * I ∈ G} ∧
  ComplexAnalysis.R2.Harmonic_R2_eps (fun p => (f (p.1 + p.2 * I)).im) {p : ℝ
    × ℝ | p.1 + p.2 * I ∈ G}
```

$$\Delta \operatorname{Re}(f) = 0, \quad \Delta \operatorname{Im}(f) = 0 \text{ on } G$$

- **HarmonicConjugate_C_eps** ($u, v : \mathbb{C} \rightarrow \mathbb{R}$, $G \subset \mathbb{C}$): A real-valued function v is a harmonic conjugate of a real-valued function u on an open set G if both are harmonic on G and they satisfy the Cauchy-Riemann equations on G .

```
def HarmonicConjugate_C_eps (u v : ℂ → ℝ) (G : Set ℂ) : Prop :=
  ComplexAnalysis.R2.Harmonic_R2_eps (fun p => u (p.1 + p.2 * I)) {p : ℝ × ℝ
    | p.1 + p.2 * I ∈ G} ∧
  ComplexAnalysis.R2.Harmonic_R2_eps (fun p => v (p.1 + p.2 * I)) {p : ℝ × ℝ
    | p.1 + p.2 * I ∈ G} ∧
  ∃ (ux uy vx vy : ℂ → ℝ),
    (∀ z ∈ G, HasPartialDerivX_C_to_R_eps u (ux z) z) ∧
    (∀ z ∈ G, HasPartialDerivY_C_to_R_eps u (uy z) z) ∧
    (∀ z ∈ G, HasPartialDerivX_C_to_R_eps v (vx z) z) ∧
    (∀ z ∈ G, HasPartialDerivY_C_to_R_eps v (vy z) z) ∧
    (∀ z ∈ G, ux z = vy z ∧ uy z = -vx z)
```

$$\Delta u = 0, \quad \Delta v = 0, \quad u_x = v_y, \quad u_y = -v_x \text{ on } G$$

3 ComplexAnalysis/Sarason/Chapter1.lean

- **chordalMetric** ($z, w \in \mathbb{C} \rightarrow \mathbb{R}$): The chordal distance between two points in the complex plane mapped to the Riemann sphere.

```
noncomputable def chordalMetric (z w : ℂ) : ℝ :=
  (2 * Complex.abs (z - w)) / (Real.sqrt (1 + Complex.normSq z) * Real.sqrt (1
    + Complex.normSq w))
```

$$\chi(z, w) = \frac{2|z - w|}{\sqrt{1 + |z|^2} \sqrt{1 + |w|^2}}$$

- **chordalMetricToInf** ($z \in \mathbb{C} \rightarrow \mathbb{R}$): The chordal distance from a point z to the point at infinity ∞ .

```
noncomputable def chordalMetricToInf (z : ℂ) : ℝ :=
  2 / Real.sqrt (1 + Complex.normSq z)
```

$$\chi(z, \infty) = \frac{2}{\sqrt{1 + |z|^2}}$$

- **Φ (Stereographic Projection)**

```
noncomputable def Φ (z : ℂ) : EuclideanSpace ℝ (Fin 3) :=
  ![ (2 * z.re) / (1 + Complex.normSq z),
    (2 * z.im) / (1 + Complex.normSq z),
    (Complex.normSq z - 1) / (1 + Complex.normSq z) ]
```

$$\Phi(z) = \left(\frac{2\operatorname{Re}(z)}{1+|z|^2}, \frac{2\operatorname{Im}(z)}{1+|z|^2}, \frac{|z|^2-1}{1+|z|^2} \right) \in S^2$$

- **ChordalC**: Type alias for $\mathbb{C}$ equipped with the chordal metric.

```
def ChordalC : Type := C
```

ChordalC = $\mathbb{C}$ (equipped with chordal metric)

4 ComplexAnalysis/Sarason/Chapter2.lean

- **del** (∂)

```
def del (f : C → C) (z : C) : C :=
  let L := fderiv ℝ f z
  let df_dx := L 1
  let df_dy := L I
  (1 / 2 : C) * (df_dx - I * df_dy)
```

```
/-- The  $\partial$  operator (del-bar) for a function  $f : \mathbb{C} \rightarrow \mathbb{C}$  at point  $z$ . -/
```

$$\partial f = \frac{1}{2} \left(\frac{\partial f}{\partial x} - i \frac{\partial f}{\partial y} \right)$$

- **delBar** ($\bar{\partial}$)

```
def delBar (f : C → C) (z : C) : C :=
  let L := fderiv ℝ f z
  let df_dx := L 1
  let df_dy := L I
  (1 / 2 : C) * (df_dx + I * df_dy)
```

```
-- Local notation for Sarason's flavor
```

```
local notation f "z" => del f
```

```
local notation f "zbar" => delBar f
```

```
/--
```

```
  Relating Sarason's Weierstrass derivative to Mathlib's Filter-based
  HasDerivAt.
```

```
-/
```

$$\bar{\partial} f = \frac{1}{2} \left(\frac{\partial f}{\partial x} + i \frac{\partial f}{\partial y} \right)$$

- **del_eps** / **delBar_eps** ($u_{x0}, u_{y0}, v_{x0}, v_{y0} \in \mathbb{R}$): Wirtinger derivatives expressed purely in terms of the real partial derivatives of $f = u + iv$.

```
noncomputable def del_eps (ux uy vx vy : ℝ) : C :=
  (1 / 2 : C) * ( (ux + vy) + I * (vx - uy) )
```

$$\partial f = \frac{1}{2}(u_{x0} + v_{y0}) + \frac{i}{2}(v_{x0} - u_{y0})$$

- **delBar_eps**

```
noncomputable def delBar_eps (ux uy vx vy : ℝ) : C :=
  (1 / 2 : C) * ( (ux - vy) + I * (vx + uy) )
```

$$\bar{\partial}f = \frac{1}{2}(u_{x0} - v_{y0}) + \frac{i}{2}(v_{x0} + u_{y0})$$

- **path_in_C** ($x, y \in \mathbb{C}$): A continuous path $\gamma : [0, 1] \rightarrow \mathbb{C}$ starting at x and ending at y .

```
def path_in_C (x y : ℂ) : Type := Path x y

/-- Velocity / derivative of a path  $\gamma : \text{path\_in\_C } x y$  at parameter time  $t \in \mathbb{R}$  . -/
```

$$\gamma : [0, 1] \rightarrow \mathbb{C} \text{ where } \gamma(0) = x, \gamma(1) = y$$

- **translated_path** ($c, x, y \in \mathbb{C}, \gamma : \text{path_in_C } x y$): A path shifted by the constant c , creating a path from $c + x$ to $c + y$.

```
noncomputable def translated_path {x y : ℂ} (c : ℂ) (γ : path_in_C x y) :
  path_in_C (c + x) (c + y) where
  toFun := fun t => c + γ.toFun t
  continuous_toFun := Continuous.add continuous_const γ.continuous_toFun
  source' := by { change c + γ.toFun 0 = c + x; have h : γ.toFun 0 = x := (γ :
    Path x y).source'; rw [h] }
  target' := by { change c + γ.toFun 1 = c + y; have h : γ.toFun 1 = y := (γ :
    Path x y).target'; rw [h] }
```

$$\gamma_c(t) = c + \gamma(t)$$

- **pathDeriv** ($\gamma : \text{path_in_C } x y, t \in \mathbb{R}$): The tangent vector $\gamma'(t) \in \mathbb{C}$, extracted via choice from **HasDerivAt_R_to_C_eps** extended to $\mathbb{R}$.

```
noncomputable def pathDeriv {x y : ℂ} (γ : path_in_C x y) (t : ℝ) : ℂ :=
  deriv_R_to_C_eps γ.extend t

/-- The direction of a path  $\gamma$  at time  $t$  is the argument of its derivative. -/
```

$$\gamma'(t) = \frac{d\gamma}{dt}$$

- **pathDirection** ($\gamma : \text{path_in_C } x y, t \in \mathbb{R}$): The argument (angle) of the tangent vector, $\arg(\gamma'(t))$.

```
noncomputable def pathDirection {x y : ℂ} (γ : path_in_C x y) (t : ℝ) : ℝ :=
  arg (pathDeriv γ t)

/-- The angle between two paths  $\gamma_1$  and  $\gamma_2$  at times  $t_1$  and  $t_2$  is the difference
  of their directions. -/
```

$$\text{Direction}(\gamma, t) = \arg(\gamma'(t))$$

- **pathAngle** ($\gamma_1, t_1, \gamma_2, t_2$): The angle between two intersecting paths.

```
noncomputable def pathAngle {x1 y1 x2 y2 : ℂ} (γ1 : path_in_C x1 y1) (t1 : ℝ) (
  γ2 : path_in_C x2 y2) (t2 : ℝ) : ℝ :=
  pathDirection γ2 t2 - pathDirection γ1 t1

/--
  The angle between two paths is equal to the argument of the product of the
  second path's derivative
  and the conjugate of the first path's derivative, modulo  $2\pi$ .
  We prove this equality here assuming the angle difference stays within the
  principal branch bounds.
- /
```

$$\text{Angle}(\gamma_1, t_1, \gamma_2, t_2) = \arg(\gamma_2'(t_2)) - \arg(\gamma_1'(t_1))$$

- **conformal** ($f : \mathbb{C} \rightarrow \mathbb{C}, z_0 \in \mathbb{C}$): f preserves angles between any two paths intersecting at z_0 with non-zero derivatives.

```
def conformal (f : ℂ → ℂ) (z₀ : ℂ) : Prop :=
  ∀ (x₁ y₁ x₂ y₂ : ℂ) (γ₁ : path_in_ℂ x₁ y₁) (t₁ : ℝ) (γ₂ : path_in_ℂ x₂ y₂) (t₂ : ℝ) (d₁ d₂ : ℂ),
    γ₁.extend t₁ = z₀ →
    γ₂.extend t₂ = z₀ →
    HasDerivAt_R_to_ℂ_eps γ₁.extend (pathDeriv γ₁ t₁) t₁ →
    HasDerivAt_R_to_ℂ_eps γ₂.extend (pathDeriv γ₂ t₂) t₂ →
    pathDeriv γ₁ t₁ ≠ 0 →
    pathDeriv γ₂ t₂ ≠ 0 →
    -- HasDerivAt_R_to_ℂ_eps f f' t says f' is the derivative of f at t.
    HasDerivAt_R_to_ℂ_eps (f ∘ γ₁.extend) d₁ t₁ →
    HasDerivAt_R_to_ℂ_eps (f ∘ γ₂.extend) d₂ t₂ →
    d₁ ≠ 0 ∧ d₂ ≠ 0 ∧ arg d₂ - arg d₁ = pathAngle γ₁ t₁ γ₂ t₂

/--
  \SII.11 Angle preservation at a point.  CONFORMALITY.
  If f is holomorphic at z₀ with non-zero derivative, then it preserves angles
  between regular paths intersecting at z₀.
-/
```

Conformal f at $z_0 \iff f$ preserves angles between all regular intersecting curves at z_0

- **centered_line_path** ($z, v \in \mathbb{C}$): A straight-line path from $z - v$ to $z + v$ evaluating to z at $t = 1/2$.

```
noncomputable def centered_line_path (z v : ℂ) : path_in_ℂ (z - v) (z + v)
  where
  toFun := fun t => z + (2 * (t : ℝ) - 1) * v
```

$$\gamma(t) = z + (2t - 1)v$$

5 ComplexAnalysis/Sarason/Chapter3.lean

- **translationMatrix** ($b \in \mathbb{C}$):

```
noncomputable def translationMatrix (b : ℂ) : GL (Fin 2) ℂ :=
  Units.mk0 !![1, b; 0, 1] ...
```

$$\begin{pmatrix} 1 & b \\ 0 & 1 \end{pmatrix}$$

- **scalingMatrix** ($a \in \mathbb{C}, a \neq 0$):

```
noncomputable def scalingMatrix (a : ℂ) (ha : a ≠ 0) : GL (Fin 2) ℂ :=
  Units.mk0 !![a, 0; 0, 1] ...
```

$$\begin{pmatrix} a & 0 \\ 0 & 1 \end{pmatrix}$$

- **inversionMatrix**:

```
noncomputable def inversionMatrix : GL (Fin 2) ℂ :=
  Units.mk0 !![0, 1; 1, 0] ...
```

$$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$